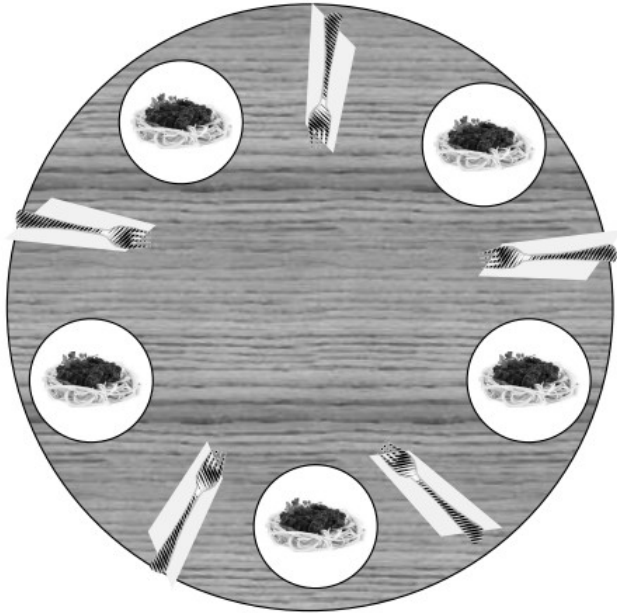


Thread in concorrenza



IL PROBLEMA DEI FILOSOFI



- 5 filosofi sono seduti attorno a un tavolo circolare; ogni filosofo ha un piatto di spaghetti tanto scivolosi che necessitano di 2 forchette per poter essere mangiati; sul tavolo vi sono in totale 5 forchette.
- Ogni filosofo ha un comportamento ripetitivo, che alterna due fasi:
 - una fase in cui pensa,
 - una fase in cui mangia.

Rappresentando ogni filosofo con un thread, realizzare una politica di sincronizzazione che eviti situazioni di deadlock.

OSSERVAZIONI



- i filosofi non possono mangiare tutti insieme: ci sono solo 5 forchette, mentre ne servirebbero 10;
- 2 filosofi vicini non possono mangiare contemporaneamente perche' condividono una forchetta e pertanto quando uno mangia, l'altro e' costretto ad attendere

SOLUZIONI



Quando un filosofo ha fame:

1. prende la forchetta a sinistra del piatto
2. poi prende quella che a destra del suo piatto
3. mangia per un po'
4. poi mette sul tavolo le due forchette.

→ **Possibilita` di deadlock:** se tutti i filosofi afferrassero contemporaneamente la forchetta di sinistra, tutti rimarrebbero in attesa di un evento che non si potra` mai verificare.

Ogni filosofo verifica se entrambe le forchette sono disponibili:

- in caso affermativo, acquisisce le due forchette (in modo atomico);
- in caso negativo, aspetta.

→ in questo modo non si puo` verificare deadlock (non c'e` possesso e attesa)

Problema dei filosofi (soluzione)

Associo ad ogni bacchetta un semaforo (binario)

Semaphore b[5]; // array di semafori

Semafori sono inizializzati a 1

b[i] = new Semaphore(1);

....

b[4] = new Semaphore(1);

Il filosofo i, prima di iniziare a mangiare, acquisisce in sequenza:

la bacchetta i (bacchetta sx)

la bacchetta (i+1)%5 (bacchetta dx)

Un filosofo non può prendere prima la bacchetta di destra invece di quella di sinistra!

Per quale motivo
effettivamente
questo funziona?

Algoritmo produttore/consumatore

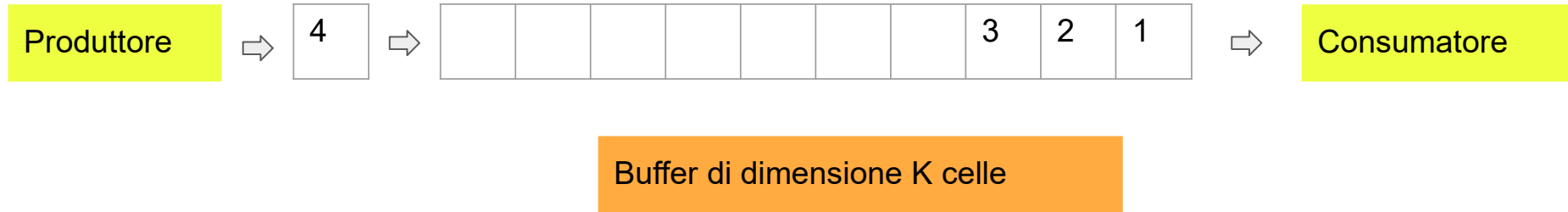
Il **problema del produttore-consumatore** detto anche problema del buffer limitato è un problema dove ci sono due processi che condividono un buffer comune di dimensione fissata dove:

- Un processo produttore inserisce dati all'interno,
- Un processo consumatore legge i dati prodotti, rimuovendoli di volta in volta dal buffer.

Problema:

- Come assicurare che il produttore si fermi di produrre quando il buffer è pieno e riprenda l'esecuzione quando almeno un blocco di memoria è libero?
- Come assicurare che il consumatore non cerchi dati quando il buffer è vuoto e venga avvisato quando qualche dato è disponibile?

Problema Produttore/Consumatore



Implementazione dell'algoritmo tramite un array circolare

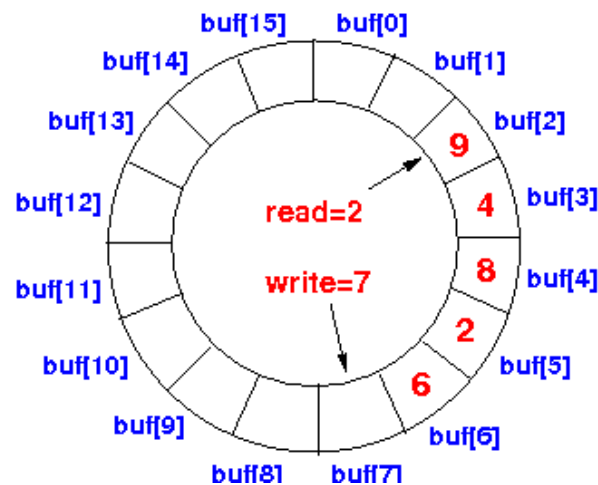
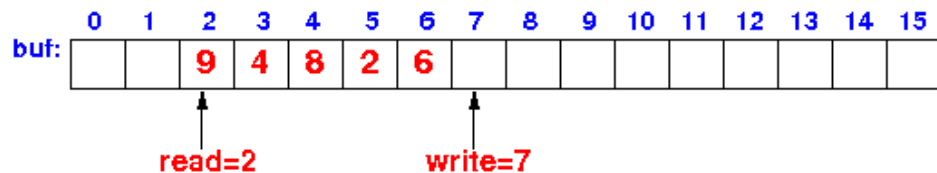
Un array circolare è una struttura dati dove ogni elemento dell'array c'è una continuità logica.

In pratica dopo aver scritto nell'ultimo elemento si riparte dal primo.

Quindi un produttore può continuare a scrivere come se l'array fosse infinito, utilizzando l'operatore di modulo per riportarsi in una posizione effettiva dell'array

es. `buf[posiz%dim]=7` // scrivo 7 in una posizione precisa dell'array.

Se `posiz` è 64 e `dim` è 16 come nell'esempio scriverò in quale posizione 7?



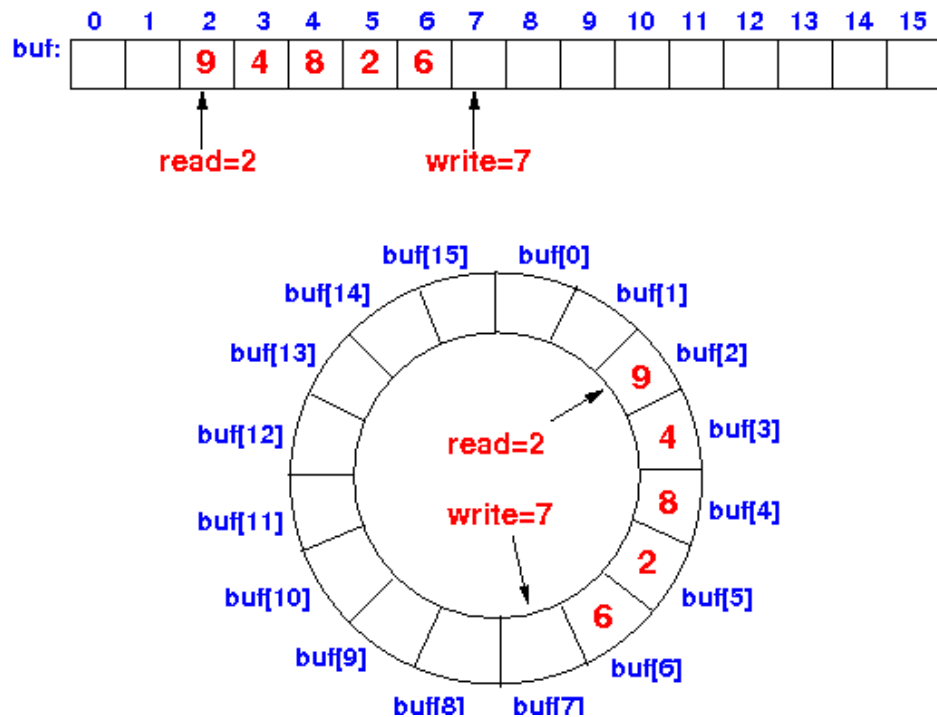
Implementazione dell'algoritmo tramite un array circolare

- Come assicurare che il produttore si fermi di produrre quando il buffer è pieno?

Attraverso un semaforo che dichiara se l'array circolare è pieno. Quando il consumatore lo avrà liberato di almeno una cella di memoria il produttore potrà scrivere

- Come assicurare che il consumatore non cerchi dati quando il buffer è vuoto?

Attraverso un semaforo che dichiara se l'array circolare è vuoto. Quando il produttore lo avrà riempito di almeno una cella di memoria il consumatore potrà leggere.



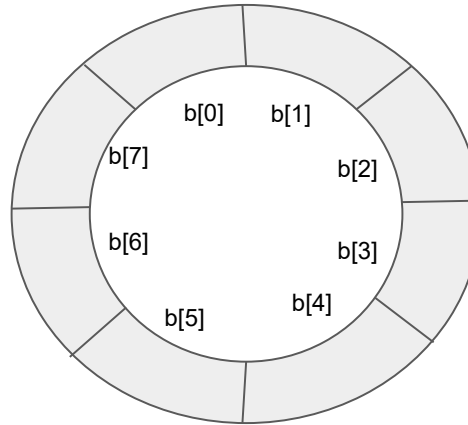
Operazioni su un array circolare

Dato il seguente array



b[0] b[1] b[2] b[3] b[4] b[5] b[6] b[7]

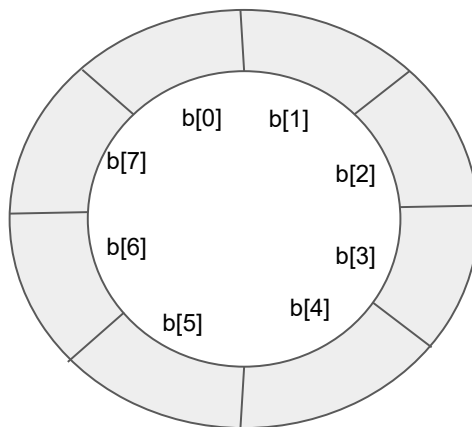
Può essere visto così...



Operazioni su un array circolare

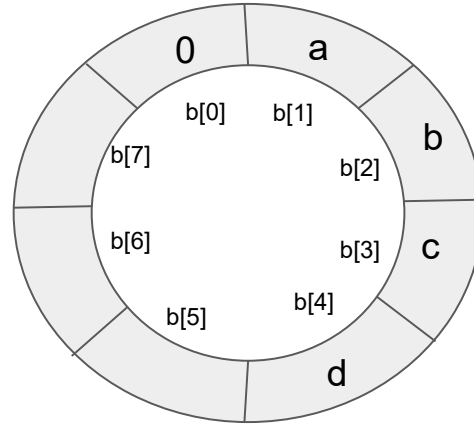
La posizione successiva di $b[i]$ è
 $b[i+1 \% \text{DIM}]$

Se ad esempio $i=7$
il successivo è $b[i+1 \% \text{DIM}]$
 $= b[0]$

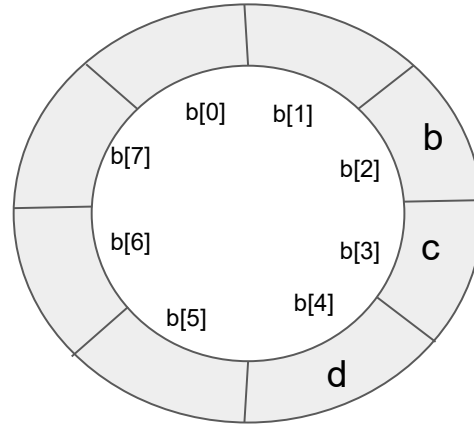


Operazioni su un array circolare

Inserisco 5 elementi

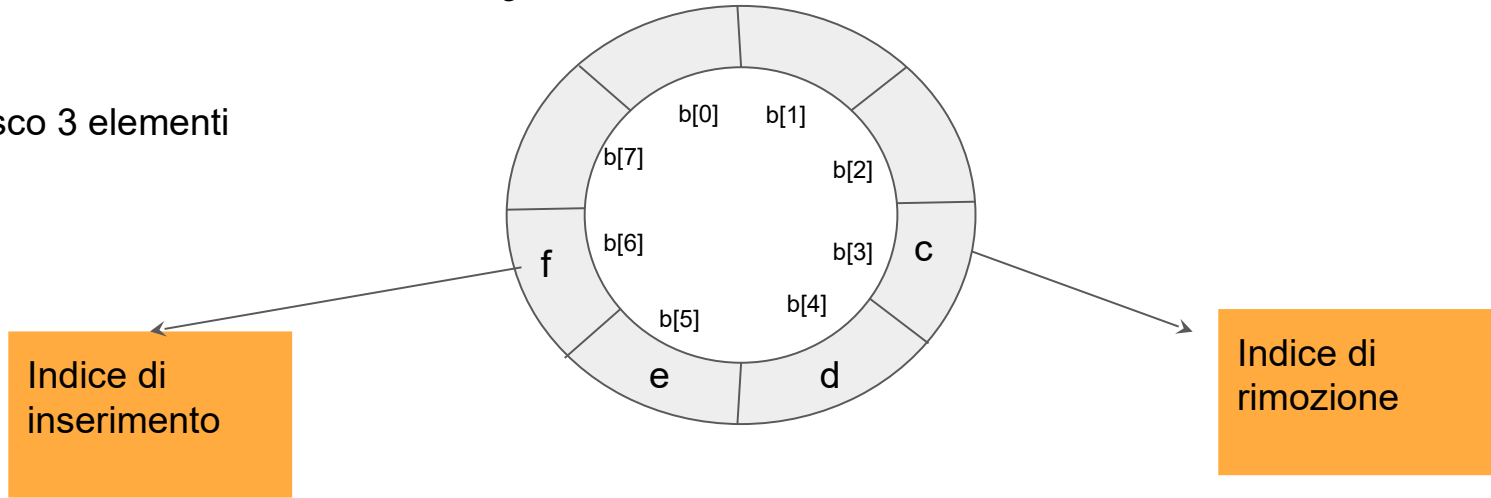


Rimozione di due elementi

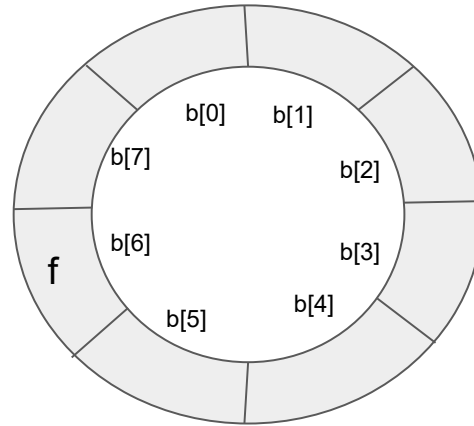


Operazioni su un array circolare

Inserisco 3 elementi



Rimozione di tre elementi



La classe BufferCircolare - tipo Integer

```
public class BufferCircolare{  
    public int N;  
    public int B[];  
    public int in=0; // elemento da inserire  
    public int out=0; // elemento da estrarre
```

```
}
```

metodi

```
void inserisci(int Elem) {
```

```
    b[in] = elem;
```

```
    in=(in+1)%N;
```

```
}
```

```
int rimuovi() {
```

```
    tmp=B[out];
```

```
    out=(out+1)%N;
```

```
    return tmp;
```

```
}
```

Schema di soluzione con i semafori

Partendo dalla classe Buffer
Circolare uso due semafori

N = Dimensione del buffer
Due semafori su due variabili
P = Numero posizioni piene
V = Numero posizioni vuote
Dove $P=0$ e $V = N$ all'inizio!

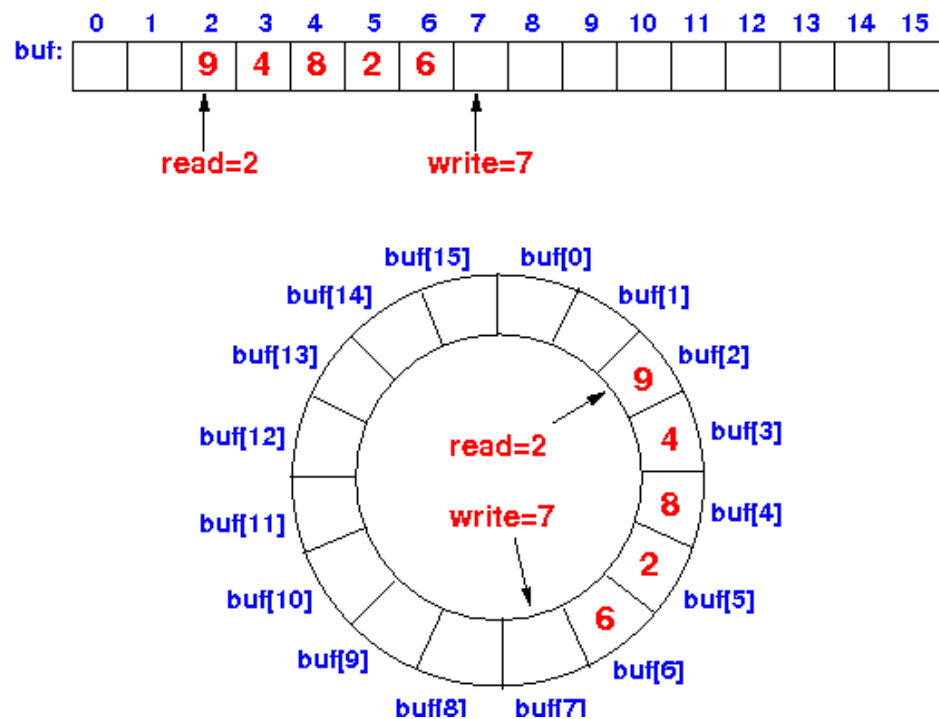
```
Produttore (inserisco un valore K)
Down(V)
Inserisco K
UP(P)
```

```
Consumatore (estraggo un valore J)
Down(P)
Estraggo J
UP(v)
Uso J
```

Se non ci sono posizioni vuote o non ci sono elementi da estrarre questo schema è bloccante....

Bozza di soluzione

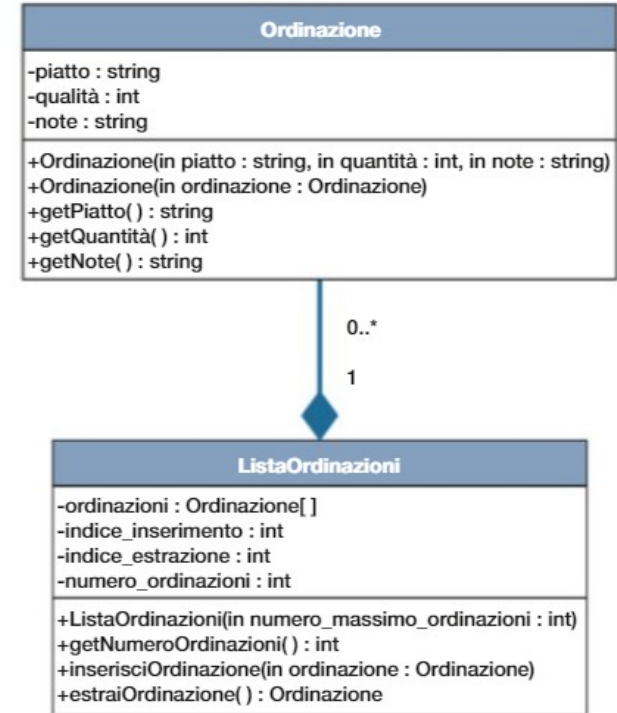
```
public void inserisci(int elem){  
    posVuote.acquire();  
    buff [index]=elem;  
    index=(index+1) %N;  
    posPiene.release();  
}  
  
public int estrai(){  
    posPiene.acquire();  
    int tmp = buff [out];  
    out=(out+1) %N;  
    posVuote.release();  
}
```



Esercizio produttore-consumatore



- In un piccolo ristorante fast-food le ordinazioni si presentano alla cassa posta all'ingresso e la cucina le prende in carico una alla volta, nell'ordine in cui sono state effettuate. Il cassiere predispone per ogni piatto ordinato un foglietto che ne riporta il nome, la quantità ordinata e le eventuali note di preparazione e lo pone in cima alla pila delle ordinazioni precedenti; il cuoco quando è disponibile prende il primo foglietto in fondo alla pila e ne inizia la preparazione. L'informatizzazione del processo descritto può essere basata sulle classi del diagramma UML a fianco.
- La classe **ListaOrdinazioni** deve realizzare uno schema produttore/consumatore con politica FIFO di oggetti di tipo **Ordinazioni**.
- Implementare le classi **ListaOrdinazioni** e **Ordinazioni**. Per verificare il corretto funzionamento dell'implementazione realizzare due classi che simulino il comportamento del cassiere e del cuoco in modo che rappresentino due thread distinti che, rispettivamente, inseriscono ed estraono ordinazioni dalla lista con un tempo di attesa casuale tra un'operazione e la successiva.



Algoritmo lettori scrittori

Problema:

- Più thread condividono l'accesso ad una memoria condivisa.

Alcuni thread sono interessati solo a leggere l'area di memoria e non c'è alcuna necessità di consumare una volta sola i dati (i dati possono essere letti piu' volte)

- Altri thread devono scrivere (modificare) l'area

Algoritmo lettori scrittori

Esempio: un database molto grande con tanti processi in competizione per leggere o scrivere

Molti possono leggere contemporaneamente, ma se un processo sta scrivendo (modifica) nessuno può leggere!

Algoritmo lettori scrittori

Il problema consiste nel sincronizzare l'accesso alla risorsa condivisa in modo che valgano i vincoli:

- 1) Nessun thread deve poter accedere ai dati, nè per scrivere nè per leggere, se è in corso un'operazione di scrittura, (ci sarebbe una corruzione dei dati)
- 2) più thread lettori **potrebbero** poter leggere contemporaneamente se non è in corso una scrittura

Soluzione

- Se non è necessario che piu' thread leggano dati contemporaneamente è sufficiente un unico semaforo dove un unico thread in lettura o scrittura può accedere ai dati condivisi.
- Se è necessario che piu' thread possano leggere in contemporanea servono due semafori.
 - il primo per gestire la scrittura esclusiva di un unico thread
 - il secondo per garantire la lettura ad n Thread

Soluzione (cenni)

```
writer() {  
    lockscrittura.down();    //Blocco l'area di memoria per la scrittura, se ci sono lettori è bloccante  
    // scrittura della memoria  
    lockscrittura.up()      //La rilascio in modo che i lettori possano accedere  
}
```

```
reader() {  
    locklettura.down()    //sezione critica sui lettori  
    readcount++;         //Aumenta il numero di lettori attivi  
    if (readcount == 1)  //Se è il primo lettore attivo  
        lockscrittura.down(); // blocco gli scrittori (o attendo che termini di scrivere lo scrittore)  
    locklettura.up();     //  
  
    //lettura  
  
    locklettura.down()    //sezione critica sui lettori  
    readcount--;         //un lettore se ne va  
    if (readcount == 0)  //se è l'ultimo lettore  
        lockscrittura.up(); //abilito i lettori a leggere  
    locklettura.up();     //  
}
```

ESERCIZIO 28/9



Scrivi un programma che **implementi il problema lettori scrittori**. Attraverso una classe buffer circolare di DIM elementi (con DIM = 10).

Nota bene: puoi usare quella già fatta e riadattarla nell'esercizio precedente.

Devi creare due classi:

- La classe Scrittori ha un metodo genera() che riempie il buffer tramite di dati raccolti da un file input.txt (allegato alla seguente consegna)
- La classe Lettori avrà un metodo usa() che legge il buffer ed effettua le elaborazioni necessarie a calcolare quanto scritto sotto.

Nel file input.txt sono contenute due informazioni, il codice di un sensore e la temperatura rilevata.

Esempio

A:10.5

B:11.5

Significa che ci sono cinque sensori, A e B... dove A rileva una temperatura di 10.5 gradi e B rileva una temperatura di 11.5 gradi ecc.

Nelle classi:

- La classe lettori può leggere solo tanti dati quanto è la DIM del buffer. Terminato questo tempo attende un tempo casuale tra 1 e 10 ms e poi riprova ad accedere, in questo tempo il buffer è lasciato libero.
- La classe scrittori dopo aver scritto DIM valori sul file si interrompe per un tempo casuale compreso tra 5 e 10 ms. Se non può accedere al buffer deve comunque attendere da 5 a 10 ms ulteriori e riprovare.
- Se il file è terminato, dopo un paio di secondi il programma deve terminare (puoi fermare i thread scrittori con i metodi imparati l'anno scorso).

Nel main:

- esiste un solo scrittore che è l'unico ad accedere al file,
- esistono 5 lettori associati in 5 thread associati a 5 sensori (A, B, C, D, E). L'elaborazione deve calcolare la temperatura massima rilevata da un sensore rispetto ai dati attualmente presenti nel buffer ed essa deve essere stampata opportunamente.

Le classi lettori e scrittori sono sincronizzate con l'algoritmo presente nelle slide. Un solo scrittore può scrivere l'area di memoria a patto che nessun lettore sta leggendo. Se uno o più lettori stanno leggendo i dati possono continuare a farlo in tempo indefinito, fintantoché non hanno terminato.